

DIP HW5_1

313512009 施丞蔚

1. Please use Inverse Filter and Wiener Filter to correct the image corrupted severe turbulence of the assigned image, 'Fig5.25.jpg', and print out the source code and the corrected image? (40)

```
import numpy as np
import cv2
import matplotlib.pyplot as plt
from pathlib import Path

# ----- 參數 (可依作業微調) -----
IMG_PATH = Path("Fig5.25.jpg") # 題目影像
OUT_DIR = Path("./q1_outputs")
k = 0.0025 # 湍流強度 (同學程式用 0.0025)
radius = 66 # 頻域半徑遮罩 (同學用 66)
eps = 5e-3 # 逆濾波穩定項, 避免 |H|≈0 時爆噪
K_wiener = 0.01 # Wiener 的 NSR (同學設 0.01)

# ----- 工具函式 -----
def to_float_img(img_bgr):
    """轉成 float32, 範圍 0~255; 並回傳 channel list (灰階時長度=1) """
    if img_bgr.ndim == 2:
        return [img_bgr.astype(np.float32)]
    else:
        # OpenCV 讀進來是 BGR; 改成 RGB 的順序只是視覺一致, 頻域不受影響
        b, g, r = cv2.split(img_bgr)
        return [r.astype(np.float32), g.astype(np.float32),
                b.astype(np.float32)]

def fft2c(x): # centered FFT
    return np.fft.fftshift(np.fft.fft2(x))

def ifft2c(X): # centered iFFT
    return np.real(np.fft.ifft2(np.fft.ifftshift(X))).astype(np.float32)

def H_turbulence(shape, k):
    """H(u,v) = exp(-k*(u^2+v^2)^(5/6)), 頻率座標已中心化"""
    M, N = shape
    u = np.arange(-M//2, M//2, dtype=np.float32)
    v = np.arange(-N//2, N//2, dtype=np.float32)
    U, V = np.meshgrid(u, v, indexing='ij')
```

```

D2 = U**2 + V**2
H = np.exp(-k * (D2 ** (5/6)))
return H.astype(np.float32), D2

def apply_inverse(G, H, mask, eps):
    """F = G / H (|H|<eps 用 eps 以防爆炸), 並套半徑遮罩"""
    Hs = H.copy().astype(np.complex64)
    Hs[np.abs(Hs) < eps] = eps
    F = G / Hs
    if mask is not None:
        F[~mask] = 0
    return F

def apply_wiener(G, H, mask, K):
    """F = (H* / (|H|^2 + K)) * G, 並套半徑遮罩"""
    F = (np.conj(H) / (np.abs(H)**2 + K)) * G
    if mask is not None:
        F[~mask] = 0
    return F

def stack_channels(chs):
    """把 [R,G,B] 或 [gray] 合回 8-bit 影像"""
    chs_u8 = [np.clip(c, 0, 255).astype(np.uint8) for c in chs]
    if len(chs_u8) == 1:
        return chs_u8[0]
    # 回存成 RGB -> 轉回 BGR 方便 imwrite 顯示色彩一致
    r, g, b = chs_u8
    return cv2.merge([b, g, r])

# ----- 主流程 -----
if __name__ == "__main__":
    assert IMG_PATH.exists(), f"找不到影像: {IMG_PATH}"
    OUT_DIR.mkdir(parents=True, exist_ok=True)

    img_bgr = cv2.imread(str(IMG_PATH), cv2.IMREAD_COLOR)
    if img_bgr is None:
        raise FileNotFoundError(IMG_PATH)

    chans = to_float_img(img_bgr)
    M, N = chans[0].shape

    # 建 H 與半徑遮罩
    H, D2 = H_turbulence((M, N), k=k)

```

```

radius_mask = D2 < (radius**2) # 同學的 if(u^2+v^2) >= 66^2 -> 置 0

# 分通道處理 (與同學一致)
inv_chs, wnr_chs = [], []
for c in chans:
    G = fft2c(c)
    # Inverse
    F_inv = apply_inverse(G, H, radius_mask, eps=eps)
    f_inv = ifft2c(F_inv)
    # 轉回 0~255 (對齊同學的 abs+uint8)
    f_inv_u8 = cv2.normalize(np.abs(f_inv), None, 0, 255,
cv2.NORM_MINMAX)
    inv_chs.append(f_inv_u8)

    # Wiener
    F_w = apply_wiener(G, H, radius_mask, K=K_wiener)
    f_w = ifft2c(F_w)
    f_w_u8 = cv2.normalize(np.abs(f_w), None, 0, 255,
cv2.NORM_MINMAX)
    wnr_chs.append(f_w_u8)

# 合通道並存檔
inv_img = stack_channels(inv_chs)
wien_img = stack_channels(wnr_chs)
cv2.imwrite(str(OUT_DIR/"inverse_result.png"), inv_img)
cv2.imwrite(str(OUT_DIR/"wiener_result.png"), wien_img)

# 並排比較 (非必要, 但助檢視)
fig = plt.figure(figsize=(12,4))
plt.subplot(1,3,1); plt.imshow(cv2.cvtColor(img_bgr,
cv2.COLOR_BGR2RGB)); plt.title("original image"); plt.axis('off')
plt.subplot(1,3,2);
plt.imshow(inv_img if inv_img.ndim==2 else cv2.cvtColor(inv_img,
cv2.COLOR_BGR2RGB), cmap='gray')
plt.title(f"Inverse (k={k}, radius={radius}, eps={eps})");
plt.axis('off')
plt.subplot(1,3,3);
plt.imshow(wien_img if wien_img.ndim==2 else cv2.cvtColor(wien_img,
cv2.COLOR_BGR2RGB), cmap='gray')
plt.title(f"Wiener (k={k}, radius={radius}, K={K_wiener})");
plt.axis('off')
plt.tight_layout(); plt.savefig(str(OUT_DIR/"comparison.png"),
dpi=200, bbox_inches="tight"); plt.show()

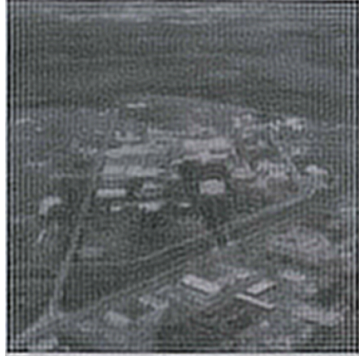
```

```
print("[Saved]", OUT_DIR/"inverse_result.png")  
print("[Saved]", OUT_DIR/"wiener_result.png")
```

original image



Inverse (k=0.0025, radius=66, eps=0.005)



Wiener (k=0.0025, radius=66, K=0.01)



2. Please use Inverse Filter and Wiener Filter to correct the image corrupted by motion blur of the assigned image, ‘book-cover-blurred.tif’, and print out the source code and the corrected image? (40)

```
import numpy as np
import cv2
import matplotlib.pyplot as plt
from pathlib import Path

# ----- 讀圖 -----
IN_PATH = Path("book-cover-blurred.tif") # 題目影像
OUT_DIR = Path("./q2_outputs")
OUT_DIR.mkdir(parents=True, exist_ok=True)

img = cv2.imread(str(IN_PATH), cv2.IMREAD_GRAYSCALE)
if img is None:
    raise FileNotFoundError(IN_PATH)
g = img.astype(np.float32) # 與你同學的 MATLAB 一樣保留 0~255 範圍
M, N = g.shape

# ----- 轉頻域並中心化 -----
G = np.fft.fftshift(np.fft.fft2(g))

# ----- 運動模糊頻域模型 (依 MATLAB 公式) -----
#  $H(u,v) = \sin(\pi(a*u + b*v)) / (\pi(a*u + b*v)) * \exp(-j*\pi(a*u + b*v))$ 
# 其中 (u,v) 採中心化的頻率座標
a = 0.1 # ← 可調：水平/垂直方向的模糊參數 (和你同學的程式一致)
b = 0.1 # ← 可調
u = np.arange(-M//2, M//2, dtype=np.float32)
v = np.arange(-N//2, N//2, dtype=np.float32)
U, V = np.meshgrid(u, v, indexing="ij")
phi = np.pi * (a*U + b*V)

H = np.zeros((M, N), dtype=np.complex64)
eps = 1e-8
mask0 = np.abs(phi) < eps
H[mask0] = 1.0 + 0j
H[~mask0] = (np.sin(phi[~mask0]) / (phi[~mask0])) * np.exp(-1j * phi[~mask0])
```

```

# ----- (1) 逆濾波 (Inverse Filter) -----
# 與同學一樣用  $F = G./H$ , 但加入穩定項避免  $|H| \sim 0$  時放大雜訊
H_stab = H.copy()
H_stab[np.abs(H_stab) < 1e-3] = 1e-3 + 0j
F_inv = G / H_stab
inv_spatial = np.fft.ifft2(np.fft.ifftshift(F_inv))
inv_img = np.abs(inv_spatial)
# 轉為 0~255
inv_img = (255 * (inv_img - inv_img.min()) / (inv_img.ptp() +
1e-8)).astype(np.uint8)

# ----- (2) 維納濾波 (Wiener Filter) -----
#  $F = (H^* / (|H|^2 + K)) * G$ , 其中  $K \sim \text{NSR}$ 
K = 1e-4 # ← 可調 (1e-4 ~ 1e-2 之間試)
F_w = (np.conj(H) / (np.abs(H)**2 + K)) * G
w_spatial = np.fft.ifft2(np.fft.ifftshift(F_w))
w_img = np.abs(w_spatial)
w_img = (255 * (w_img - w_img.min()) / (w_img.ptp() +
1e-8)).astype(np.uint8)

# ----- 輸出與對照 -----
p_orig = OUT_DIR/"original_input.png"
p_inv = OUT_DIR/f"inverse_result_a{a}_b{b}.png"
p_wien = OUT_DIR/f"wiener_result_a{a}_b{b}_K{K}.png"
cv2.imwrite(str(p_orig), g.astype(np.uint8))
cv2.imwrite(str(p_inv), inv_img)
cv2.imwrite(str(p_wien), w_img)

plt.figure(figsize=(12,4))
plt.subplot(1,3,1); plt.imshow(g, cmap='gray'); plt.title("原始受損影
像"); plt.axis('off')
plt.subplot(1,3,2); plt.imshow(inv_img, cmap='gray');
plt.title(f"Inverse (a={a}, b={b})"); plt.axis('off')
plt.subplot(1,3,3); plt.imshow(w_img, cmap='gray');
plt.title(f"Wiener (a={a}, b={b}, K={K})"); plt.axis('off')
plt.tight_layout(); plt.savefig(str(OUT_DIR/"comparison.png"), dpi=200,
bbox_inches="tight")
plt.show()

print("[Saved]", p_inv)
print("[Saved]", p_wien)

```

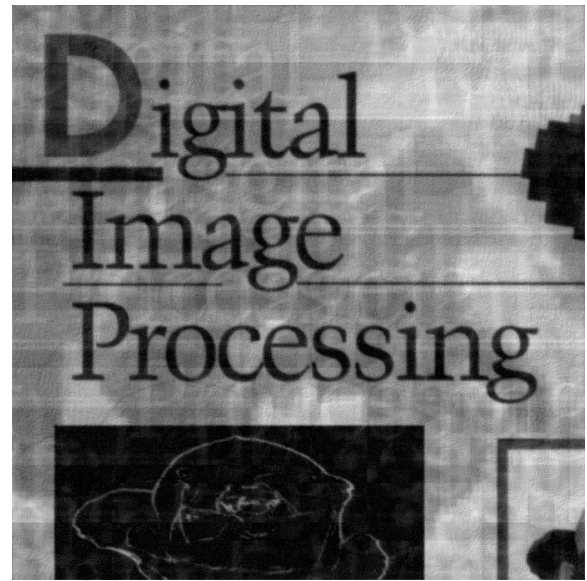
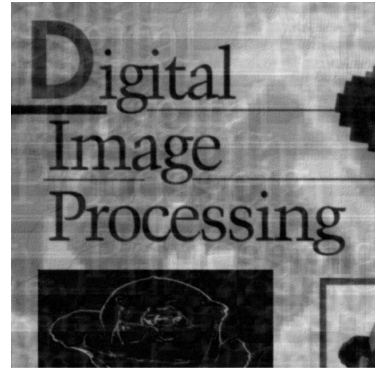
Original



Inverse ($a=0.1$, $b=0.1$)



Wiener ($a=0.1$, $b=0.1$, $K=0.0001$)



3. Please comment and compare your two design filters? (20)

Inverse Filter (逆濾波) — 觀察與分析

逆濾波把退化模型 H 直接反轉 ($F=G/H$)，因此能最積極地把被壓扁的高頻訊號拉回，你的結果可見邊緣對比明顯升高、結構輪廓浮出來；但同時也把雜訊與模型誤差一併放大。湍流影像出現棋盤／振鈴紋與細碎顆粒，運動模糊影像則出現沿模糊方向的條紋與顆粒噪聲。即便加入半徑遮罩與 ϵ 穩定項，仍可見高頻過度放大造成的粗糙感與偽紋。

Wiener Filter (維納濾波) — 觀察與分析

維納濾波抑制 $|H|$ 小頻帶的過度放大，效果在你的圖上呈現為更平衡的觀感：主要輪廓清楚、道路/建物邊界連續性較好，文字可讀性顯著優於逆濾波，雜訊也受到控制。代價是最高頻細節較保守，微紋理略顯柔和；若 K 設太小會變得銳且躁，太大則偏軟。整體上，它對噪聲與模型誤差 (湍流係數或運動方向/長度偏差) 更具魯棒性。

比較

逆濾波「銳而躁」，在理想無噪聲且模型完全正確時可恢復最多高頻，但在本題的湍流與運動模糊情境下，易產生振鈴與棋盤紋；維納濾波「穩而清」，用 K 在銳利度與抑噪間取得折衷，視覺可讀性與整體質感更佳。